



National University of Computer & Emerging Sciences,  
Karachi



Fall 2026, Lab Manual

## Introduction to Object-Oriented Programming (Lab)

|               |                 |
|---------------|-----------------|
| Course code : |                 |
| Instructor :  | Shahzaib Shaikh |
|               |                 |

### Lab Title

### Classes & Objects

### Lab Activities

- ❖ Implementing the base `Account` class with `__init__`
- ❖ Creating multiple `Account` objects
- ❖ Inspecting instance attributes

### Lab Objectives

By the end of this lab, students should be able to:

- ❖ Explain the relationship between classes and objects.
- ❖ Define a Python class using the `class` keyword.
- ❖ Create a constructor using `__init__`.
- ❖ Initialize instance attributes.
- ❖ Create multiple objects from the same class.
- ❖ Access and inspect object attributes.

## Prerequisites

Students should have a basic understanding of:

- Python variables and data types
- Functions and parameters
- Basic Python syntax

## What is a Class?

A **class** is a blueprint or template used to create objects.

A class describes:

- What data an object will contain
- What operations or behaviors an object can perform

**For example**, consider a bank account.

A bank account may contain:

- Account holder name
- Account number
- Balance

Instead of creating separate variables for every account, we can define an `Account` class and create multiple account objects from it.

## Real-World Example

Think about a **Student ID Card**.

The design of the ID card is like a **class**.

Each student's actual ID card is an **object**.

**The design may specify:**

```
Student Name  
Roll Number  
Department  
Semester
```

Every student's card follows the same design, but the information on each card is different.

# Class in OOP

## Bank Account Example

A class is a **blueprint** or **template** used to create **objects**.

It describes what data an object will **contain** and what **operations** it can perform.

### Account Class (Blueprint)

**Data (Attributes)**

- account\_holder\_name
- account\_number
- balance

**Operations (Behaviors)**

- deposit()
- withdraw()
- check\_balance()

## Objects (Accounts)

### Account Object 1



Name: Ali Khan  
Acc No: 1001  
Balance: 50,000

deposit()  
withdraw()  
check\_balance()

### Account Object 2



Name: Sara Ahmed  
Acc No: 1002  
Balance: 75,000

deposit()  
withdraw()  
check\_balance()

### Account Object 3



Name: John Smith  
Acc No: 1003  
Balance: 20,000

deposit()  
withdraw()  
check\_balance()

## Real-World Example

Think about a Student ID Card.

- The design of the ID card is like a class.
- Each student's actual ID card is an object.
- The design may specify:
  - Student Name
  - Roll Number
  - Department
  - Semester

### StudentIDCard Class (Blueprint)



Name : \_\_\_\_\_  
Roll No : \_\_\_\_\_  
Department: \_\_\_\_\_  
Semester : \_\_\_\_\_



## Objects (Student ID Cards)

### Student Card 1



Name : Ahmed Raza  
Roll No : 21CS01  
Dept : Computer Science  
Semester: 5



### Student Card 2



Name : Ayesha Malik  
Roll No : 21CS02  
Dept : Computer Science  
Semester: 5



### Student Card 3



Name : Bilal Hassan  
Roll No : 21CS03  
Dept : Computer Science  
Semester: 5



Just like many students have ID cards with the same design but different information, many account objects are created from the same Account class.

## Creating a Class Step by Step

### Step 1: Define a Class

In Python, we create a class using the **class** keyword.

### Syntax

## Creating a Class in Python

### 1. Syntax

Indentation is required for the class body.

```
class ClassName:
    # class body
```

**class** is a keyword used to define a class.

**ClassName** is the name of the class (use PascalCase).

Class body contains attributes (data) and methods (functions).

### 2. Example

```
class Account:
    pass
```

This creates a class named **Account**.

Class Name

**Account**

Currently it has no content (only a placeholder).

**pass**  
(Placeholder)



The **pass** statement is used as a placeholder. It tells Python: "Do nothing for now, we will add code here later."

## Important Point

By convention, class names in Python are usually written using **PascalCase**.

### Examples:

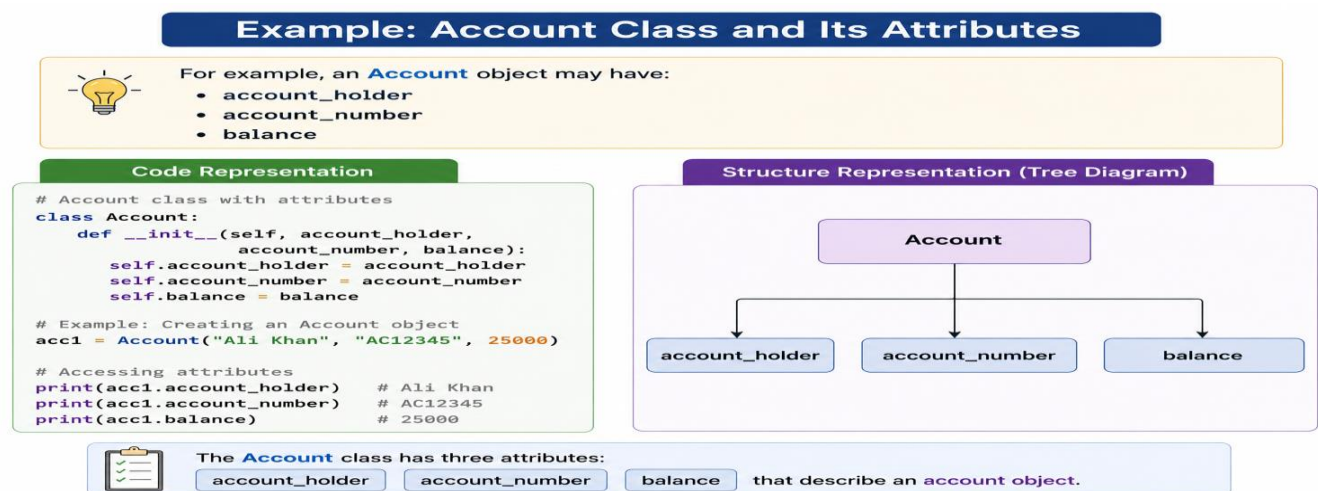
1. Account
2. Student
3. BankAccount
4. Employee
5. Car
6. MobilePhone

## Understanding Attributes

What is an Attribute?

An **attribute** is a piece of data associated with an object.

For example, an **Account** object may have:



However, we need a way to give values to these attributes when an object is created.

This is where the **constructor** is used.

## What is a Constructor?

### Understanding the `__init__()` Method

The `__init__()` method is a special method in Python.

It is automatically executed when an object is created.

It is commonly called the **constructor** because it is used to initialize the newly created object's attributes.

For example:

```
def __init__(self, account_holder,  
             account_number, balance):  
    self.account_holder = account_holder  
    self.account_number = account_number  
    self.balance = balance
```

When we create an object:

```
# Example: Creating an Account object  
acc1 = Account("Ali Khan", "AC12345", 25000)
```

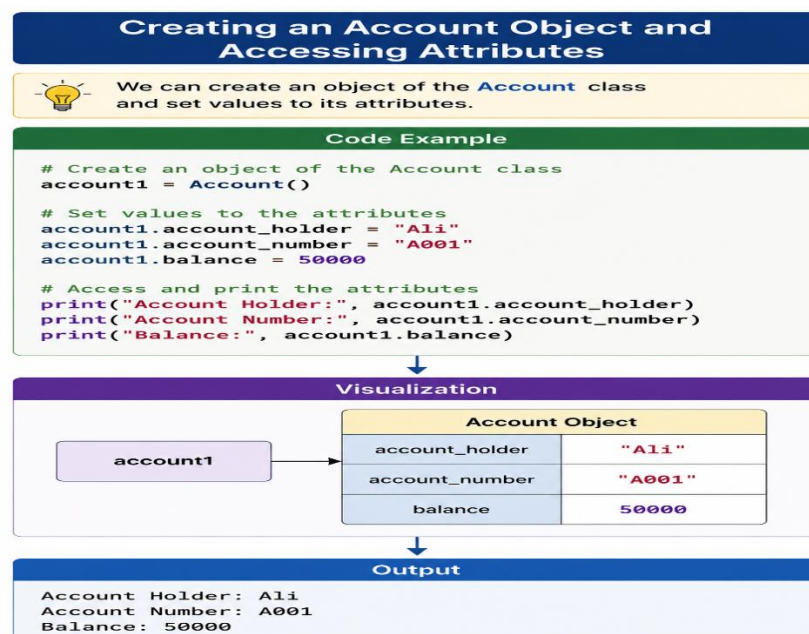
Python will automatically calls:

```
__init__()
```

### Why Do We Use a Constructor?

A constructor allows us to **initialize an object's data** at the time the **object is created**.

For example, instead of creating an empty account and assigning values separately:



We can initialize everything when creating the object:

```
# Example: Creating an Account object  
acc1 = Account("Ali Khan", "AC12345", 25000)
```

This makes our code:

- Cleaner
- Easier to understand
- More organized
- Less repetitive

## Understanding `self`

### What is `self`?

`self` refers to the **current object**.

Consider:

```
def __init__(self, account_holder,  
              account_number, balance):  
    self.account_holder = account_holder  
    self.account_number = account_number  
    self.balance = balance
```

The left side:

```
self.account_holder
```

represents the attribute belonging to the current object.

The right side:

```
account_holder
```

represents the value received by the constructor.

`self` represents the current object.

## Accessing Object Attributes

We can access an object's attributes using the **dot (.) operator**.



## Accessing Object Attributes



### Access an Object's Attributes

We can access an object's attributes using the **dot (.)** operator.

#### 1. Access Account Holder

```
print(account1.account_holder)
```



Output:

Ali

#### 2. Access Account Number

```
print(account1.account_number)
```



Output:

A001

#### 3. Access Balance

```
print(account1.balance)
```



Output:

50000

#### Complete Example

```
# Create an object of the Account class and set values
account1 = Account()
account1.account_holder = "Ali"
account1.account_number = "A001"
account1.balance = 50000

# Access and print attributes
print(account1.account_holder)
print(account1.account_number)
print(account1.balance)
```

Output:

Ali  
A001  
50000

## Inspecting Instance Attributes

Using `__dict__`

Python provides `__dict__` to inspect an object's instance attributes.

For example:

### Viewing Object Attributes Using `__dict__`



The `__dict__` attribute returns a **dictionary** containing all attributes and their values of an object.

For account1

```
print(account1.__dict__)
```



Possible output:

```
{'account_holder': 'Ali', 'account_number': 'A001', 'balance': 50000}
```

➡ This allows us to see the attributes stored inside the object.

For account2

```
print(account2.__dict__)
```



Possible output:

```
{'account_holder': 'Sara', 'account_number': 'A002', 'balance': 75000}
```

Note:



Notice that both objects have the **same attribute names** but **different values**.

## Relation between Class and Object:

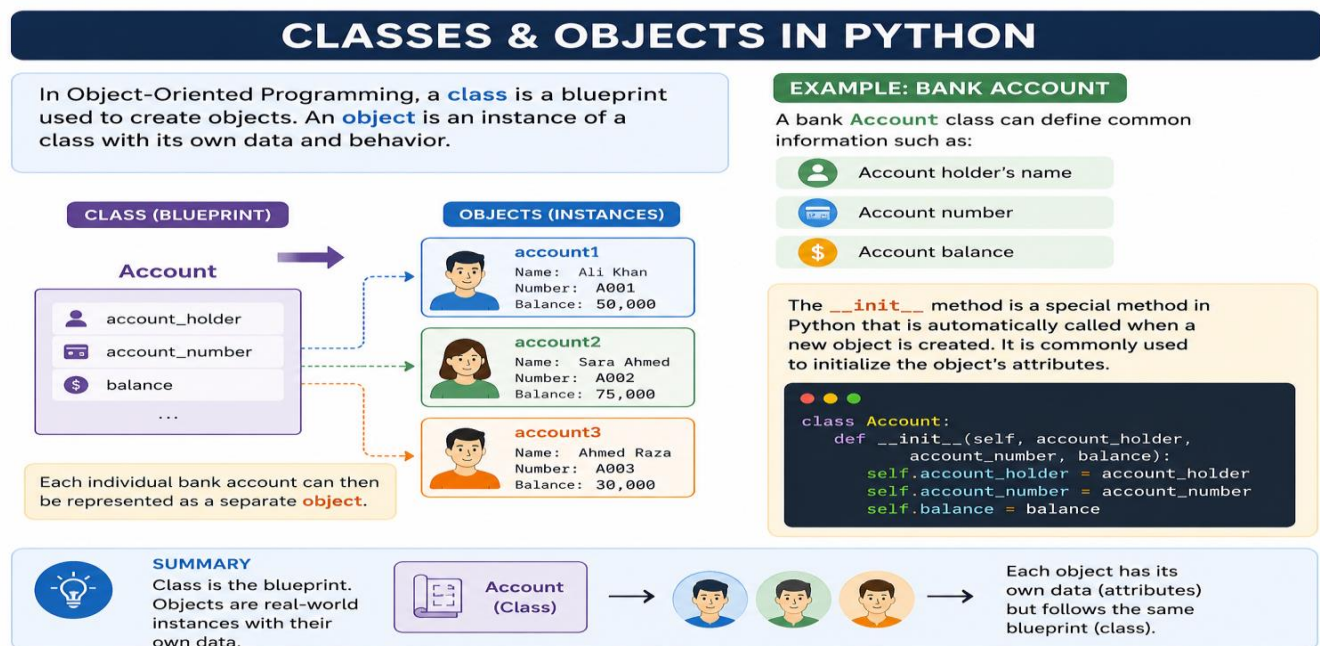
In Object-Oriented Programming, a **class** is a blueprint used to create objects. An **object** is an instance of a class with its own data and behavior.

For example, a bank Account class can define common information such as:

- Account holder's name
- Account number
- Account balance

Each individual bank account can then be represented as a separate object.

The **`__init__`** method is a special method in Python that is automatically called when a new object is created. It is commonly used to initialize the object's attributes.



Complete Lab Practical Example:

### Created Class Names as Account with Constructor

3 usages

```

class Account:
    def __init__(self, account_holder, account_number, balance):
        self.account_holder = account_holder
        self.account_number = account_number
        self.balance = balance

```



## Create Three Object

```
# Creating objects
account1 = Account( account_holder: "Ali", account_number: "A001", balance: 50000)
account2 = Account( account_holder: "Sara", account_number: "A002", balance: 75000)
account3 = Account( account_holder: "Ahmed", account_number: "A003", balance: 30000)
```

## Accessing Attributes of Object one

```
# Accessing attributes
print("Account 1")
print("Holder:", account1.account_holder)
print("Number:", account1.account_number)
print("Balance:", account1.balance)

print()
```

## Accessing Attributes of Object two

```
print("Account 2")
print("Holder:", account2.account_holder)
print("Number:", account2.account_number)
print("Balance:", account2.balance)

print()
```

## Accessing Attributes of Object three

```
print("Account 3")
print("Holder:", account3.account_holder)
print("Number:", account3.account_number)
print("Balance:", account3.balance)
```

## Inspecting instance Attributes

```
# Inspecting instance attributes
print()
print(account1.__dict__)
```

## Important Concepts to Remember

### Key Concepts in Python OOP

| Concept                                                                                                          | Meaning                                     |
|------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
|  <b>Class</b>                   | Blueprint used to create objects            |
|  <b>Object</b>                  | An instance of a class                      |
|  <b>Attribute</b>               | Data stored in an object                    |
|  <b><code>__init__()</code></b> | Method used to initialize object attributes |
|  <b><code>self</code></b>       | Refers to the current object                |
|  <b><code>.</code></b>          | Used to access an object's attributes       |
|  <b><code>__dict__</code></b>   | Shows an object's instance attributes       |

**Class** → Blueprint

**Object** → Instance of the blueprint

**Constructor** → Initializes the object

**Attribute** → Data belonging to the object

## Final Lab Tasks

### Task 1 — Student Class

Create a class named `Student`.

The class should have:

- Student name
- Roll number
- Department
- Semester

Create three student objects and display all attributes of each student.

### Task 2 — Employee Class

Create an `Employee` class with:

- Employee name
- Employee ID
- Department
- Salary

Create at least three employee objects.

Display the information of each employee separately.

### Task 3 — Car Class

Create a `Car` class with:

- Brand
- Model
- Year
- Price

Methods:

`getInfo()`

Create three different car objects and display their attributes.

### Task 4 — Book Class

Create a class `LibraryBook`:

- Title
  - Author
  - isIssued
- 
- Methods:
    - `issueBook()`
    - `returnBook()`
    - `displayStatus()`

Create two or more book objects and display their information.

Also use `__dict__` to inspect the attributes of each object.

### Task 5 — University Semester Course Class

Create a `semester_Course` class with:

- Course code
- Course name

- Credit hours
- Semester

Methods:

totalFees(): //which will be calculated on basis of credit hours

Create at least three course objects.

Display all attributes of each course and use `__dict__` to inspect the instance attributes.

## Task 6 — PlayList

Create a class Playlist:

- PlayListTitle
- Songs []
- Methods:
  - Add\_songs() // this will add songs to given playlist

Create two or more book objects and display their information.

Also use `__dict__` to inspect the attributes of each object.